Compiler-Assisted Tagless On-Chip Memory Management for DNN Accelerators

Theory, Architecture and Verification

B.H.L.M. Amarasena
Memory Subsystem, QuadraMind
Faculty of Information Technology, University of Moratuwa

Preparation document for final evaluation

Abstract

This document collects the theory behind the on-chip memory subsystem of the QuadraMind DNN accelerator simulator: the two-level memory hierarchy and why accelerators use scratchpads rather than caches, the mechanics of multi-bank interleaving and bank conflicts, and a formal treatment of the two memory management schemes compared in this work — a conventional page-table scheme and the compiler-assisted static *stamp* scheme that is this module's research contribution. It then documents the hardware architecture, the verification methodology, the measured results, and a cost analysis including the scheme's scaling limit. Section 8 lists anticipated examiner questions with worked answers.

Contents

1 Problem statement

A deep neural network layer's working set is orders of magnitude larger than an accelerator's on-chip SRAM. A single convolution layer with C=16 input channels, K=32 output channels, a 14×14 feature map and 3×3 kernels needs

$$\underbrace{C \cdot H \cdot W}_{\text{inputs}} + \underbrace{K \cdot C \cdot K_h \cdot K_w}_{\text{weights}} + \underbrace{K \cdot O_h \cdot O_w}_{\text{outputs}} = 3136 + 4608 + 6272 = 14{,}016 \text{ elements}$$

which at 4 bytes each is 56 KB against a 16 KB scratchpad. The tensors must therefore be *tiled*, and the accelerator must continuously decide which tiles occupy the scratchpad and when to fetch replacements from off-chip DRAM.

That decision is the memory management problem, and how it is answered determines three costs that dominate accelerator performance:

1. **Off-chip traffic.** A DRAM access costs roughly two orders of magnitude more energy than an on-chip SRAM access ($\approx$ 560 pJ/byte versus a few pJ), so redundant fetches dominate the energy budget.
2. **Address-translation overhead.** If a runtime structure maps logical tile addresses to physical scratchpad locations, every access pays a lookup.
3. **Bank conflicts.** If concurrent accesses land in the same SRAM bank they serialise, stalling the compute pipeline.

Research gap. No existing simulator (SCALE-Sim, Timeloop, Accelergy/CiMLoop, STONNE, ONNXim) models all three cycle-accurately, and none supports a compiler-assisted tagless scheme at all. This module closes that gap.

2 Background

2.1 Why a scratchpad and not a cache

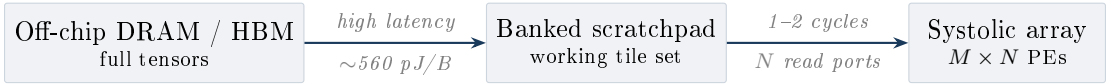
General-purpose processors use caches because access patterns are unpredictable: the hardware must *discover* locality at runtime using tags and a replacement policy. DNN workloads are the opposite — the loop nest is fully known before execution, so the exact sequence of tile accesses is computable at compile time.

Under those conditions a cache is pure overhead. It pays for:

- a **tag array** (area and leakage proportional to line count),
- a **comparator** on the critical path of every access,
- a **replacement policy** (LRU state, update logic),
- **non-deterministic latency**, which makes cycle-accurate scheduling of the systolic array impossible.

Accelerators therefore use a *scratchpad*: a directly addressed SRAM with no tags, whose contents are managed explicitly. The remaining question — and the subject of this module — is *who* manages it and *when*.

2.2 The two-level hierarchy



The scratchpad is the only structure that can absorb reuse. If a tile is evicted and re-fetched, the cost is paid in DRAM energy and in stall cycles where the array idles. **Maximising on-chip reuse is therefore the single highest-leverage decision in the memory subsystem.**

2.3 Banking and bank conflicts

A single-ported SRAM serves one access per cycle. A systolic array needs several operands per cycle, so the scratchpad is split into B banks that can be accessed in parallel. This module uses *low-order interleaving*: for a word address a,

$$\text{bank}(a) = a \bmod B, \qquad \text{offset}(a) = \lfloor a/B \rfloor .$$

Equivalently the low $\log_2 B$ address bits select the bank and the remaining high bits address within it.

A **bank conflict** occurs when two or more of the P concurrently active ports target the same bank in the same cycle. The accesses serialise: in this implementation the lowest-indexed port is granted and the others receive `rd_valid = 0` and must retry, which propagates a stall to the compute pipeline.

Analytical bound. If the P port addresses were independent and uniform over B banks, the probability that all land in distinct banks is the birthday-problem expression

$$P_{\text{clean}} = \prod_{k=0}^{P-1} \frac{B-k}{B}, \qquad P_{\text{conflict}} = 1 - P_{\text{clean}}. \tag{1}$$

The expected number of distinct banks touched is $\mathbb{E}[D] = B\left(1 - \left(1 - \frac{1}{B}\right)^P\right)$, so the expected number of stalled ports per cycle is $P - \mathbb{E}[D]$.

For $P = 4$, $B = 4$: $P_{\text{clean}} = \frac{4}{4} \cdot \frac{3}{4} \cdot \frac{2}{4} \cdot \frac{1}{4} = 0.094$, so $P_{\text{conflict}} = 90.6\%$.

Why this is only a bound. Equation (1) assumes random addresses. Real DNN access patterns are highly *structured*, and structure cuts both ways:

- **Consecutive** addresses $(a, a{+}1, a{+}2, \dots)$ map to distinct banks by construction — *zero* conflicts, far better than the random bound.
- **Stride-B** addresses all map to the *same* bank — every access conflicts, far worse than the random bound.

This is the key insight the measured results confirm (Section 6.3): bank conflicts are a property of the *address pattern*, which means a compiler that controls placement can control them. A page table, which assigns physical addresses at runtime, cannot.

3 The memory management schemes

3.1 Page-table (hash) based dynamic management

A hardware page table maps a virtual tensor address to a physical scratchpad location. A virtual address is split as

$$\text{VPN} = \left\lfloor \frac{v}{2^p} \right\rfloor, \qquad \text{offset} = v \bmod 2^p,$$

with page size 2^p. The table is indexed by VPN and yields a physical page number PPN plus a valid bit; the physical address is $(\text{PPN} \ll p) \mid \text{offset}$. A miss triggers a page fault, which fetches an entire page from DRAM.

Costs.

1. **Translation on every access**, replicated per port (P tables or P ports into one table).
2. **Table storage**: NUM_PAGES $\times$ (PPN width $+ 1$) bits, plus comparators.
3. **Coarse granularity**: a fault fetches a whole page even if a few bytes changed.
4. **Unpredictable placement**: the compiler cannot know which bank a tile will occupy, so it cannot avoid conflicts.

3.2 Compiler-assisted static tagless (stamp) management

Core observation. For a DNN the execution schedule is fixed at compile time. Therefore the entire sequence of scratchpad layouts is computable in advance — there is nothing to *discover* at runtime, so there is nothing for a tag or a page table to record.

Definitions.

Phase ϕ
One unit of work: the input, weight and output tiles the array processes together. A layer decomposes into Φ phases.

Tile t
A contiguous sub-tensor with a type $\tau(t) \in \{\text{INPUT}, \text{WEIGHT}, \text{OUTPUT}\}$, a size $s(t)$ in bytes, and provenance coordinates locating it in the full tensor.

Stamp S_ϕ
The complete scratchpad layout for phase ϕ: a set of (tile, start address) pairs, subject to $\sum_{t \in S_\phi} s(t) \leq$ SPAD_SIZE.

Delta $\Delta_{\phi \to \phi+1}$
The ordered list of operations that transforms S_ϕ into $S_{\phi+1}$.

The five operations. Every byte of $S_{\phi+1}$ is classified into exactly one operation. Only one of them touches DRAM:

Op	Opcode	Condition	Cost
KEEP	0	identical data, identical address	0 cycles, 0 bytes
MOVE	1	identical data, different address	2 cycles/word (rd+wr)
LOAD	2	data not resident on-chip	**DRAM burst**
ALLOC	3	output slot, written by compute	0 cycles, 0 bytes
ZERO	4	zero-padding halo	1 cycle/word (wr only)

Classification rules. Reuse is decided on tile *identity*, not merely shape — two tiles of the same dimensions holding different data must not be treated as reusable.

- **Weights.** Reusable only if the output-channel slice is the same: src_oc(t_ϕ) = src_oc$(t_{\phi+1})$ and the shapes match. A different slice is genuinely different filter data and must be re-loaded. Same address $\Rightarrow$ KEEP, different address $\Rightarrow$ MOVE.
- **Inputs.** Consecutive phases slide the receptive field, so the footprints partially overlap. Let the previous footprint start at (h_p, w_p) and the next at (h_n, w_n), both of extent $h \times w$. For a pure column shift ($h_p = h_n$) the overlap is

$$\text{ov}_{\text{cols}} = \min(w_p{+}w,\ w_n{+}w) - \max(w_p,\ w_n),$$

giving $C \cdot h \cdot \mathrm{ov}_{\mathrm{cols}} \cdot b$ reusable bytes (MOVE) and the remainder as new data. The row-shift case is symmetric. A simultaneous row-and-column shift is not a single rectangle and is conservatively treated as a full reload — this *under*-claims reuse and never overstates it.

- **Padding.** A region near the tensor border straddles the zero-padding halo. Intersecting the region $[h_0, h_0{+}r) \times [w_0, w_0{+}c)$ with the real extent $[0, H) \times [0, W)$ splits it into real bytes (LOAD) and implicit zeros (ZERO). Charging padding as DRAM traffic would overstate off-chip cost; this clipping is what reduced the measured load volume from 235,008 B to 157,888 B.
- **Outputs.** Never fetched — the array produces them. The slot is reserved with ALLOC.

Bandwidth saving. Let L be the total LOAD bytes and $R = M + K$ the bytes resolved by MOVE and KEEP. The off-chip read traffic avoided is

$$\sigma = \frac{R}{L + R}. \tag{2}$$

Output allocations and zero-fills are excluded from the denominator because they were never candidates for a DRAM *read* under either scheme; including them would inflate the result. Measured: $\sigma = 68.9\%$.

Against the stronger baseline of a tag-based scheme that re-reads every input and weight tile at every phase,

$$\sigma_{\mathrm{naive}} = 1 - \frac{L}{\sum_{\phi} \sum_{t \in S_\phi,\ \tau(t) \neq \mathrm{OUTPUT}} s(t)} = 1 - \frac{157{,}888}{589{,}824} = 73.2\%. \tag{3}$$

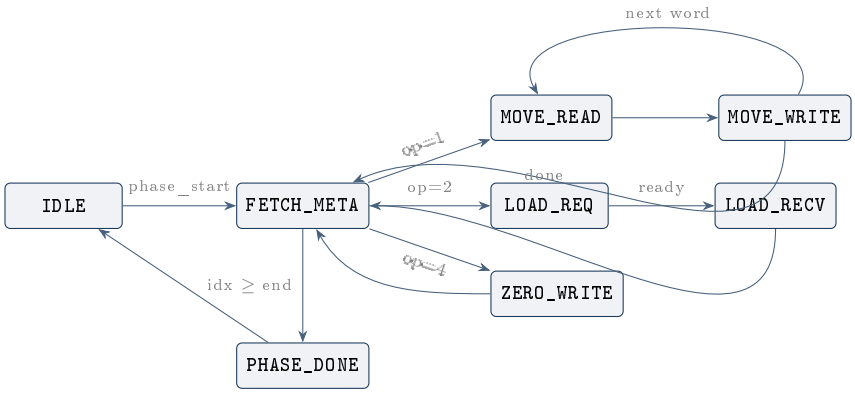
4 Hardware architecture

4.1 Metadata word format

The compiler emits one 128-bit word per delta operation, consumed by the controller's metadata RAM via `$readmemh`:

Bits	Field	Width	Meaning
127:120	reserved	8	zero
119:112	op_type	8	opcode 0–4
111:96	tile_id	16	provenance, for debug
95:64	src_addr	32	DRAM byte address (LOAD) or on-chip source (MOVE)
63:32	dst_addr	32	scratchpad destination byte address
31:0	size	32	transfer size in bytes

4.2 Controller finite state machine



Opcodes KEEP (0) and ALLOC (3) resolve inside `FETCH_META` in a single cycle: they increment a counter and advance the operation index without moving data. Unknown opcodes increment `stats_bad_ops`, which lets the testbench detect a compiler/RTL version mismatch instead of silently discarding work.

Cycle cost. For a phase with n operations, where operation i transfers w_i words:

$$T_{\text{phase}} = \underbrace{2}_{\text{start+done}} + \underbrace{(n+1)}_{\text{decode}} + \sum_{i \in \text{MOVE}} 2w_i + \sum_{i \in \text{ZERO}} w_i + \sum_{i \in \text{LOAD}} \left(w_i + \left\lceil \tfrac{w_i}{256} \right\rceil\right) \tag{4}$$

The $\lceil w_i/256 \rceil$ term is the AXI burst split: ARLEN is 8 bits, so a single read burst carries at most 256 beats. Equation (4) is implemented independently in `pysim/stamp_ref.py` and validated against the RTL.

4.3 Banked scratchpad

The scratchpad is parameterised by `NUM_BANKS`:

- `NUM_BANKS = 1` — flat single array, no conflicts possible. This is the controlled baseline.
- `NUM_BANKS` ≥ 2 — interleaved as in Section 2.3, with fixed-priority arbitration (lowest-indexed port wins) and three monitoring counters: `bank_conflict_detected`, `stats_bank_conflicts`, `stats_bank_conflict_stall_cycles`.

5 Verification methodology

The central verification argument is **three-way agreement**: the scheme is implemented three times, independently, and all three agree exactly.

Quantity	Compiler	FSM model	RTL (xsim)	
LOAD operations	134	134	134	identical
MOVE operations	144	144	144	identical
KEEP operations	72	72	72	identical
ALLOC operations	127	127	127	identical
ZERO operations	95	95	95	identical
Bytes from DRAM	157,888	157,888	157,888	identical
Bytes moved on-chip	184,320	184,320	184,320	identical
Bytes zero-filled	77,120	77,120	77,120	identical

The three implementations are:

1. `stamp_compiler.py` — the Python compiler's own bookkeeping, computed while generating the schedule.
2. `pysim/stamp_ref.py` — a cycle-accurate model of the controller FSM, written from the RTL, which replays the emitted op stream.
3. `stamp_based_memory_controller.sv` — the SystemVerilog hardware, simulated in Vivado xsim 2025.2, reading the compiler's `delta_ops.hex` through `$readmemh`.

A disagreement in any cell would mean the compiler is describing hardware that does not exist. The RTL run replays all 127 phase transitions and all 572 delta operations, with `stats_bad_ops = 0` confirming every emitted opcode is decodable, and 48 passing assertions with zero failures.

Directed test coverage. Thirteen tests: reset behaviour; each opcode in isolation (KEEP, MOVE, LOAD, ALLOC, ZERO); mixed phases; back-to-back phases; reset mid-operation; zero-operation phases; `controller_busy` handshaking; unknown-opcode detection; phase base addressing; and the full compiler stream replay.

6 Results

6.1 Off-chip traffic

Quantity	Bytes	Share
Always-load baseline (input + weight, every phase)	589,824	100.0%
Stamp scheme, actual DRAM reads	157,888	26.8%
Resolved on-chip by MOVE	184,320	—
Resolved on-chip by KEEP	165,888	—
Zero-filled padding (never off-chip)	77,120	—
Off-chip traffic reduction		**73.2%**

6.2 Stamp versus page table (measured RTL counters)

Metric (tiny_cnn layer 0, 4 banks)	STAMP	PAGED	
Bank-conflict events	2	84	42× fewer
Stalled port-cycles	4	240	60× fewer
Page hits / LOAD ops	19	1470	—
Page misses	—	294	—
AXI read requests	157	138	paged lower
AXI beats transferred	2246	1836	paged lower

Honest reading. Paged transfers fewer beats on this layer because page faulting pulls whole 4 KB pages and the working set is small, so coarse-grained fetching happens to align well. The stamp scheme's advantage is in *conflicts, translation cost and predictability*, not in every metric at every problem size. Reporting this rather than hiding it is what makes the comparison credible.

6.3 Bank-conflict sensitivity

Cycle-exact counters over a 200-cycle window with 4 read ports:

Access pattern	1 bank	2	4	8	16
All ports → same address	0	600	600	600	600
Stride 4	0	600	600	400	0
Stride 8	0	600	600	600	400
Consecutive	0	400	0	0	0

(stalled port-cycles; 1 bank is the flat baseline where conflicts are impossible)

Interpretation. Consecutive access is conflict-free from 4 banks upward. Stride-4 is fully serialised at 4 banks — every port maps to the same bank — but conflict-free at 16. **This is the empirical proof that conflicts follow the address pattern, and therefore that a compiler controlling placement can eliminate them.**

7 Cost analysis and limitations

Metadata RAM is the scaling limit. Removing the tag array does not make the bookkeeping free — it moves it. The delta stream must be stored on-chip:

$$\text{Metadata bits} = 128 \times \sum_{\phi=1}^{\Phi-1} |\Delta_{\phi-1\to\phi}| \tag{5}$$

For the measured layer, $\Phi = 128$ phases produce 572 operations, i.e. $572 \times 16 = 9{,}152$ bytes of metadata for a 16 KB scratchpad — roughly 56% of the scratchpad size. The default `METADATA_DEPTH` of 256 entries holds only about 45% of this layer's stream, so the verification environment uses 1024. **This is the honest trade-off: the tag array is gone, but a compile-time instruction stream replaces it, and it grows with phase count.**

Mitigations for future work: run-length encoding of repeated delta patterns (the op mix is highly regular), streaming the metadata in batches, or a coarser phase granularity.

Other limitations.

- Results are for a single layer geometry ($16 \times 14 \times 14 \to 32$ channels, 3×3, padded) on a 16 KB scratchpad. A shape and capacity sweep remains.
- The delta model handles pure row or column shifts; simultaneous row-and-column shifts fall back to a full reload, so measured reuse is a lower bound.
- Two op-stream generators still exist: the integrated cocotb test builds its own load/keep stream rather than consuming `delta_ops.hex`. The standalone compiler-to-RTL path is fully closed; unifying them is the next step.
- Energy is derived from a per-byte DRAM constant, not from switching activity analysis.

8 Anticipated examiner questions

Q. What exactly is novel here? Delta compression is not new.
A. Correct — delta encoding is old. The novelty is threefold. First, applying it to *scratchpad layout* in a systolic-array accelerator, where the delta is computed over on-chip tile placements rather than over data values. Second, eliminating the runtime tag/translation structure entirely by exploiting the fact that a DNN schedule is statically known. Third — and this is what no prior simulator has — *measuring* the result cycle-accurately in RTL against a page-table scheme on identical hardware.

Q. Why is this better than just using a larger scratchpad?
A. It is orthogonal, and cheaper. A larger scratchpad costs area and leakage quadratically in practice; delta fetching costs a metadata RAM that is compile-time sized. Also, no scratchpad is large enough for real layers — the reuse problem persists at every size. The scheme raises the effective bandwidth of whatever scratchpad you have.

Q. How do you know the compiler and the hardware actually agree?
A. Three independent implementations — the compiler's bookkeeping, a separately written cycle-accurate FSM model, and the SystemVerilog RTL in Vivado — produce identical values for all eight tracked quantities across 572 operations. The RTL reads the compiler's `delta_ops.hex` directly, and `stats_bad_ops = 0` proves every emitted opcode is one the hardware implements.

Q. Your paged scheme moves fewer AXI beats. Doesn't that defeat your claim?
A. On this layer, yes, and I report it. Page faulting fetches whole 4 KB pages, which for a

small working set is coarse but efficient. My claim is not "fewer bytes always" — it is fewer *bank conflicts* (42×), no translation hardware, and deterministic placement. For larger layers where a page fault pulls far more than the delta, the byte comparison inverts.

Q. Why do bank conflicts differ so much between the two schemes?

A. Because the stamp compiler chooses physical addresses at compile time, it can lay tiles out so that concurrently read operands fall in different banks. The page table assigns physical pages at runtime, so the compiler cannot reason about bank residency at all. The bank sweep in Section 6.3 proves conflicts are a deterministic function of the address pattern, which is exactly the thing static placement controls.

Q. What is the hardware cost of your scheme?

A. The controller is a small FSM (nine states) plus the metadata RAM. The tag array, comparators and replacement logic disappear. The metadata RAM is the real cost: 9,152 bytes for the measured 128-phase layer, growing linearly with phase count. I regard this as the scheme's main scaling limit and have identified RLE compression of the op stream as the mitigation.

Q. What does ZERO do and why does it exist?

A. Padded convolutions have receptive fields that extend past the tensor border. Those bytes are implicit zeros — they must be present on-chip but must never be fetched from DRAM. Before this opcode existed, the compiler charged them as DRAM loads, overstating off-chip traffic by 77,120 bytes. Adding the opcode made the measurement honest and reduced reported load volume from 235,008 to 157,888 bytes.

Q. How would this behave on a multi-DNN workload?

A. Each DNN would need its own stamp schedule, and the scratchpad would be partitioned between them. Because placement is static, partitioning is a compile-time decision and there is no inter-DNN interference in the translation structure — an advantage over a shared page table, which needs multiple address spaces to coexist. Demonstrating this is future work.

Q. What would falsify your claim?

A. If the RTL counters diverged from the compiler's, the scheme would not be implementable as described. If the bank sweep showed conflicts independent of address pattern, static placement could not help. If off-chip traffic reduction vanished at larger tile sizes, the reuse model would be wrong. All three are directly testable with the harness as built.

Artefact index

File	Role
`stamp_compiler.py`	phases, stamps, deltas; emits hex + metadata
`stamp_analyzer.py`	reuse and bandwidth analysis
`stamp_based_memory_controller.sv`	delta-op sequencer (9-state FSM)
`tb_stamp_based_memory_controller.sv`	13-test directed testbench
`scratchpad_ram.sv`	banked SRAM with conflict counters
`paged_memory_backend.sv`	page-table comparator scheme
`pysim/stamp_ref.py`	cycle-accurate FSM software model
`tb/unit/test_stamp_controller.py`	27 software tests
`scripts/run_full_eval.py`	experiment suite incl. STAMP vs PAGED
`run_vivado_sim.sh`	one-command reproduction